Variáveis não,
Atributos (Características do objeto)
Funções não,
Métodos (Comportamento/Ação do objeto)

CLASSE:

Uma **classe** é a representação de um objeto no código. Ela reúne os atributos e métodos que representam esse objeto. Classe sendo a receita do bolo e objeto, o bolo (Sempre começando com maiúsculo e no singular).

INSTANCIAR:

Ao instanciar, o programa reserva espaço na memória para armazenar aquele objeto.

```
Pessoa pessoa = new Pessoa();
```

CONSTRUTOR:

O construtor tem o mesmo nome da classe, construtor não possui tipo de retorno.

Se você não criar nenhum construtor, o Java cria automaticamente um construtor vazio.

Por isso: `Pessoa pessoa = new Pessoa();` pode funcionar mesmo que você não tenha escrito: `public Pessoa() {}`

construtor vazio ou em branco, criado implicitamente pelo Java.

Podemos criar um construtor que recebe informações:

```
public Pessoa(String nome, int idade) {
  this.nome = nome;
  this.idade = idade;}
```

Agora podemos fazer:

```
Pessoa pessoa = new Pessoa("Marina", 31);
```

Isso permite que o objeto já seja criado com determinados valores.

Quando você cria um construtor explícito com parâmetros:

```
public Pessoa(String nome, int idade)
new Pessoa();
```

não funciona mais automaticamente, a menos que você também crie um construtor vazio.

Podemos ter vários construtores na mesma classe.

```
public Pessoa() {
  this.nome = "Usuário";
  this.idade = 0;}
public Pessoa(String nome, int idade) {
  this.nome = nome;
  this.idade = idade;}
```

Assim podemos fazer:

```
Pessoa p1 = new Pessoa();
Pessoa p2 = new Pessoa("Marina", 31);
```

Os dois são válidos porque existem dois construtores diferentes.

O THIS é uma palavra-chave usada para se referir à própria classe/objeto.

O THIS é obrigatório quando chamamos outro construtor dentro da própria classe.

```
this.nome = nome;
```

pense:

atributo = parâmetro

ENCAPSULAMENTO:

Encapsulamento é uma técnica para **ocultar/proteger atributos** e limitar o acesso a eles. A responsabilidade de definir como os dados podem ser acessados ou modificados fica dentro da própria classe

Private: Apenas na própria classe

Protected: Mesmo pacote + Subclasses em outros pacotes

Public: Qualquer lugar do projeto

GETTER:

Se o atributo é private, como outra classe consegue **ler** seu valor?

```
private String nome;
public String getNome() {
return nome; }
```

String → getNome(), int → getIdade(), boolean → isDormindo()

SETTER:

O setter serve para **alterar/modificar** um atributo.

```
public void setNome(String nome) {
  this.nome = nome;}
```

Condições:

```
public void setIdade(int idade) {
  if (idade >= 0) {
    this.idade = idade;
  } else {
    System.out.println("Idade não pode ser menor que 0");}}
```

Assim não deixamos qualquer valor entrar.

O setter pode verificar condições antes de modificar o atributo.

"Você **não precisa obrigatoriamente** criar getter e setter para todos os atributos."

Não são apenas atributos que podem ser private.

Também podemos fazer:

```
private int gerarCodigo() {}
```

Esses métodos podem funcionar como **métodos auxiliares**, utilizados apenas internamente pela classe.

A outra classe não precisa saber como o código é gerado.

ASSOCIAÇÂO DE OBJETOS:

Uma classe pode possuir uma referência para um objeto de outra classe.

Exemplo:

```
public class Disciplina {
  private String nome;
  private Professor professor;}
private Professor professor;
```

significa que uma Disciplina possui uma associação com um Professor.

Podemos então fazer:

```
Professor professor = new Professor("Marina", "74123698");
Disciplina disciplina =
  new Disciplina("POO", professor);
```

O objeto Professor foi criado e passado para Disciplina.

Uma classe pode usar métodos de outro objeto. Se Disciplina possui um Professor, ela pode utilizar os métodos desse objeto.

Por exemplo:

```
public void imprimirInfo() {
  System.out.print(
"Professor:"+this.professor.getNome()
};}
```

Aqui temos:

this.professor.getNome()

ou seja:

A Disciplina está acessando o método getNome() do objeto Professor que ela possui.

Associação e referência compartilhada

Imagine:

```
Professor professor = new Professor("Marina", "74123698");
Disciplina d1 = new Disciplina("POO", professor);
EscolaPolitecnica escola = new EscolaPolitecnica(professor);
```

As duas estruturas podem estar associadas ao **mesmo objeto Professor**.

Nesse caso, não temos dois professores independentes. Temos referências para o mesmo objeto.

Por isso, uma alteração feita nesse objeto pode ser observada pelas estruturas que estão associadas a ele.

Existe uma diferença MUITO importante entre:

Alterar o objeto existente

Substituir a referência por um novo objeto.

```
public void substituirProfessor(String nome, String codRh) {
  this.professor = new Professor(nome, codRh);
}
```

Aqui a Disciplina passa a possuir **um novo Professor**.

Isso é diferente de simplesmente alterar os atributos do professor antigo.

ARRAYLIST

Principais Características

Tamanho Dinâmico: Cresce automaticamente quando atinge a capacidade máxima.

Tipos de Dados: Guarda apenas **objetos** (classes wrapper como Integer, Double, String, ou objetos de classes personalizadas). Não aceita tipos primitivos diretos (int, double, boolean).

Acesso Rápido: Acesso direto por índice em tempo constante $O(1)$.

Inserção/Remoção: Rápida no final, mas pode ser lenta no início/meio $O(n)$, pois exige o deslocamento dos demais elementos.

Ordenação: Mantém a ordem exata de inserção dos elementos.

Java

Tipos Primitivos: Não é possível declarar ArrayList<int>. O correto é usar a Wrapper Class: ArrayList<Integer>.

Índice Fora dos Limites: Tentar acessar lista.get(10) em uma lista de tamanho 3 lança a exceção IndexOutOfBoundsException.

Remoção de Números Inteiros: Se tiver um ArrayList<Integer> e fizer lista.remove(2), o Java entenderá que você quer remover o **índice 2**, e não o número 2. Para remover o número 2, use lista.remove(Integer.valueOf(2)).

```
public static void main(String[] args) {
```